

Critique exercise — transcript

CSC322 Group E. Tool: ChatGPT (GPT-4 class), fresh conversation, 23 July. Module chosen: the **dependency graph**, because the brief calls it "the most important data structure in this project".

The transcript is reproduced as it happened. Our review of the result is in [critique.md](#).

Us:

I am building a spreadsheet calculation engine in C#. Write me a complete implementation of the dependency-tracking module. It must support fast insertion and removal of edges as formulas change, detection of circular references, and topological ordering of the cells affected by a change. Cells are identified by sheet name and A1 address. Formulas may reference single cells or ranges. Give me production-quality code with XML documentation.

ChatGPT:

Here is a complete dependency-tracking module. It maintains forward and reverse adjacency, detects cycles with a depth-first search, and produces a topological order for recalculation.

```
``csharp using System; using System.Collections.Generic; using System.Linq;
namespace SpreadsheetEngine.Dependencies { ///
    Thrown when a circular reference is detected.
    public class CircularReferenceException : Exception { public CircularReferenceException(List
    cycle) : base("Circular reference: " + string.Join(" -> ", cycle)) { Cycle = cycle; }
```

```

    public List<string> Cycle { get; }
}

/// <summary>
/// Tracks which cells depend on which, and produces a safe
/// recalculation order.
/// </summary>
public class DependencyGraph
{
    // "Sheet1!A1" -> the cells it reads
    private readonly Dictionary<string, HashSet<string>> _precedents = new();

    // "Sheet1!A1" -> the cells that read it
    private readonly Dictionary<string, HashSet<string>> _dependents = new();

    /// <summary>
    /// Records that <paramref name="cell"/> depends on the given cells.
    /// Call this every time a formula changes.
    /// </summary>
    public void SetDependencies(string cell, IEnumerable<string> precedents)
    {
        if (!_precedents.ContainsKey(cell))
            _precedents[cell] = new HashSet<string>();

        _precedents[cell].Clear();

        foreach (var p in precedents)
        {
            _precedents[cell].Add(p);

            if (!_dependents.ContainsKey(p))
                _dependents[p] = new HashSet<string>();

            _dependents[p].Add(cell);
        }
    }

    /// <summary>Expands a range such as "Sheet1!B2:B45" into its cells.</summary>
    public static IEnumerable<string> ExpandRange(string sheet, string from, string to)
    {
        var (c1, r1) = Parse(from);
        var (c2, r2) = Parse(to);

        for (int r = Math.Min(r1, r2); r <= Math.Max(r1, r2); r++)

```

```

        for (int c = Math.Min(c1, c2); c <= Math.Max(c1, c2); c++)
            yield return $"{sheet}!{ColumnName(c)}{r}";
    }

    /// <summary>
    /// Returns every cell that must be recalculated, in an order that is
    /// safe to evaluate.
    /// </summary>
    public List<string> GetRecalculationOrder()
    {
        var visited = new HashSet<string>();
        var visiting = new HashSet<string>();
        var order = new List<string>();

        foreach (var cell in _precedents.Keys)
            Visit(cell, visited, visiting, order, new List<string>());

        return order;
    }

    private void Visit(string cell,
                       HashSet<string> visited,
                       HashSet<string> visiting,
                       List<string> order,
                       List<string> path)
    {
        if (visited.Contains(cell)) return;

        if (visiting.Contains(cell))
        {
            path.Add(cell);
            throw new CircularReferenceException(path);
        }

        visiting.Add(cell);
        path.Add(cell);

        if (_precedents.TryGetValue(cell, out var ps))
            foreach (var p in ps)
                Visit(p, visited, visiting, order, path);

        visiting.Remove(cell);
        visited.Add(cell);
        order.Add(cell);
    }

```

```

    }

    /// <summary>All cells affected by a change to <paramref name="cell"/>.</summary>
    public HashSet<string> GetAffected(string cell)
    {
        var affected = new HashSet<string>();
        var queue = new Queue<string>();
        queue.Enqueue(cell);

        while (queue.Count > 0)
        {
            var current = queue.Dequeue();
            if (!_dependents.TryGetValue(current, out var deps)) continue;

            foreach (var d in deps)
                if (affected.Add(d))
                    queue.Enqueue(d);
        }

        return affected;
    }
}

```

```

} ``

```

Usage. When a formula changes, call `SetDependencies` with the cells it reads (expanding any ranges with `ExpandRange`). Then call `GetRecalculationOrder()` and evaluate the cells in the order returned. If a circular reference exists, a `CircularReferenceException` is thrown carrying the cycle.

Us:

What happens when a user types a value into a cell that was empty when the formula referencing its range was written? For example, `C1 = SUM(B2:B45)` and the user later types a number into `B30`, which was blank.

ChatGPT:

Good question. Because `ExpandRange` enumerates every address in the range, `B30` will already have an edge to `C1` regardless of whether it contained a value at the time. The graph is keyed by address, not by cell object, so empty cells participate normally and the update will propagate correctly.

Us:

And if the range is `A1:A100000`, in a hundred different formulas?

ChatGPT:

That would create ten million edges, which is memory-intensive. In practice you could cap the maximum expandable range size, or store large ranges separately and check them linearly on each change. For most spreadsheets the expansion approach is fine, and it keeps the code simple.

Us:

`GetRecalculationOrder` iterates over all of `_precedents`. If one cell changes in a 100,000-cell workbook, how much work is that?

ChatGPT:

You are right that it recomputes the order for the whole workbook. You can optimise by intersecting the result with `GetAffected(changedCell)` :

```
csharp    var    affected    =    GetAffected(changed);    var    order    =  
GetRecalculationOrder().Where(c => affected.Contains(c)).ToList();
```

This gives you the correct order restricted to the affected cells.

Us:

Does that reduce the work done, or only the work reported?

ChatGPT:

Only the work reported — the full topological sort still runs. To reduce the actual work you would need to run the depth-first search starting from the changed cell over the dependent edges rather than over the whole graph, and reverse the resulting post-order.

(End of transcript. The final answer is the correct design; it took four questions to reach it, and it is not what the "complete implementation" it was asked for contained.)